

REQUIREMENT SPECIFICATION & DESIGN DOCUMENT

Project Title: Student Attendance System - Group 12

Contents

1. Functional Requirements (FR).....	4
2. Non-Functional & Development Requirements.....	5
2.1. Non-Functional Requirements (URPS Quality Attributes).....	5
2.2. Development Requirements.....	5
3. Data Flow Diagrams (DFD).....	6
3.1. DFD Level 0 (Context Diagram).....	6
3.2. DFD Level 1.....	8
4. Use Case Diagram & Specifications.....	9
4.1. Identification of Actors.....	9
4.2. Identification of Use Cases.....	10
4.3. Use Case Diagram.....	11
4.4. Detailed Use Case Specifications.....	12
5. Class Diagram (OOP Design).....	19
5.1. Class Descriptions and Responsibilities.....	19
5.2. Class Relationship & Multiplicity Specifications.....	20
6. Data Model (Logical Database Schema).....	23
6.1. Entity Mapping Overview.....	24

6.2. Logical Table Schema Specifications.....	24
7. Interface Design Description.....	29
7.1. Screen 01 - Login.....	29
7.2. Screen 02 - Lecturer Dashboard.....	30
7.3. Screen 03 - Student Check-in.....	32
7.4. Screen 04 - Attendance Report.....	33

1. Functional Requirements (FR)

System Administration (Admin)

- FR1:** The system shall allow the administrator to manage user accounts, including creating, viewing, updating, and deactivating profiles for lecturers and students.
- FR2:** The system shall allow the administrator to manage courses, class sections, and assign students and lecturers to their respective classes.

Lecturer Operations

- FR3:** The system shall allow the lecturer to initialize an attendance session and generate a dynamic QR code and a 6-digit session PIN with a configurable active duration.
- FR4:** The system shall allow the lecturer to view real-time student check-in status and live attendance metrics during an ongoing class session.
- FR5:** The system shall allow the lecturer to manually override and update any student's attendance status (Present, Late, Absent).
- FR6:** The system shall allow the lecturer to export aggregated class attendance summary reports into downloadable spreadsheet formats (.xlsx / .csv).

Student Operations

- FR7:** The system shall allow registered users (Students, Lecturers, Admins) to authenticate securely using valid account credentials.
- FR8:** The system shall allow the student to check in to an active session by scanning the dynamic QR code or manually submitting the valid 6-digit session PIN.
- FR9:** The system shall allow the student to view personal attendance history, total absences, and attendance percentage per enrolled course.
- FR10:** The system shall allow the student to submit an absence leave request with attached justification documents for lecturer review.

2. Non-Functional & Development Requirements

2.1. Non-Functional Requirements (URPS Quality Attributes)

Usability: The system shall provide an intuitive Graphical User Interface (GUI) requiring no more than three interaction steps for standard check-in tasks.

Reliability: The system shall maintain an operational availability of at least 99% and enforce data integrity with zero data loss or duplicate attendance records.

Performance: The system shall process check-in transactions in less than 2.0 seconds and support a concurrent load of at least 100 simultaneous requests per second without latency degradation.

Security: All user passwords shall be salted and hashed using cryptographic functions (SHA-256). Generated dynamic QR codes and session PINs shall enforce a strict Time-to-Live (TTL) expiration mechanism to prevent proxy attendance.

2.2. Development Requirements

Programming Language & GUI: The core backend logic of the system will be developed using C++. C++ was strategically selected for its high execution performance, robust Object-Oriented Programming (OOP) capabilities, and efficient memory management, which are strictly required for handling real-time attendance processing and algorithmic token generation. For the presentation layer, the Qt Framework will be utilized. Qt provides a comprehensive, highly optimized set of UI libraries, ensuring a seamless, responsive, and cross-platform desktop graphical interface for both students and lecturers without compromising native system performance.

Database Management System (DBMS): SQLite for embedded standalone deployment or MySQL for central server storage.

Version Control: Source code versioning and collaborative development will be strictly managed via Git. A shared GitHub repository will serve as the central hub for code integration, branch management, issue tracking, and version history maintenance, ensuring a synchronized workflow among all group members throughout the project lifecycle.

3. Data Flow Diagrams (DFD)

3.1. DFD Level 0 (Context Diagram)

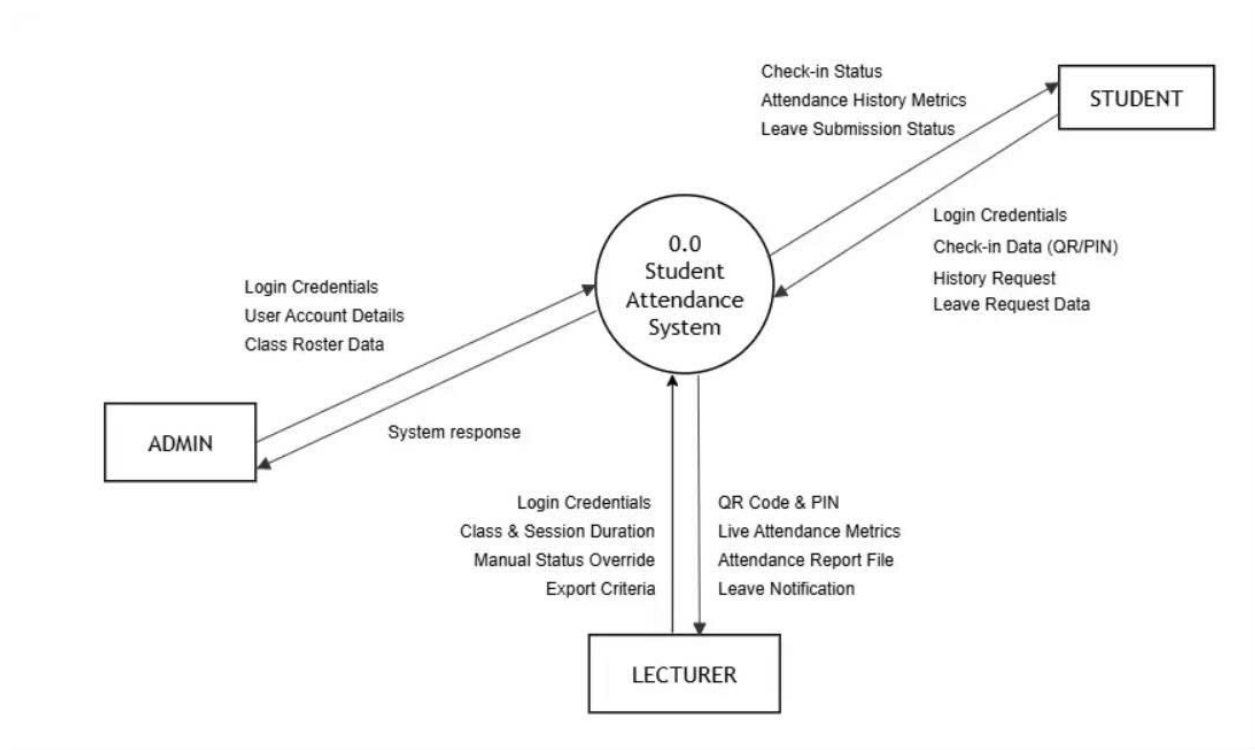


Figure 3.1: DFD Level 0 - Context Diagram of Student Attendance System

Process 0.0 (Student Attendance System): Acts as the central system boundary interacting with three external entities:

ADMIN: Supplies login credentials, user account details and class roster data; receives system confirmation responses.

LECTURER: Supplies login credentials, class session configuration, status overrides, and export filters; receives active QR/PIN tokens, live attendance metrics, export files, and leave notifications.

STUDENT: Supplies login credentials, check-in data (QR/PIN), history queries, and leave requests; receives verification status, attendance metrics, and leave submission receipts.

3.2. DFD Level 1

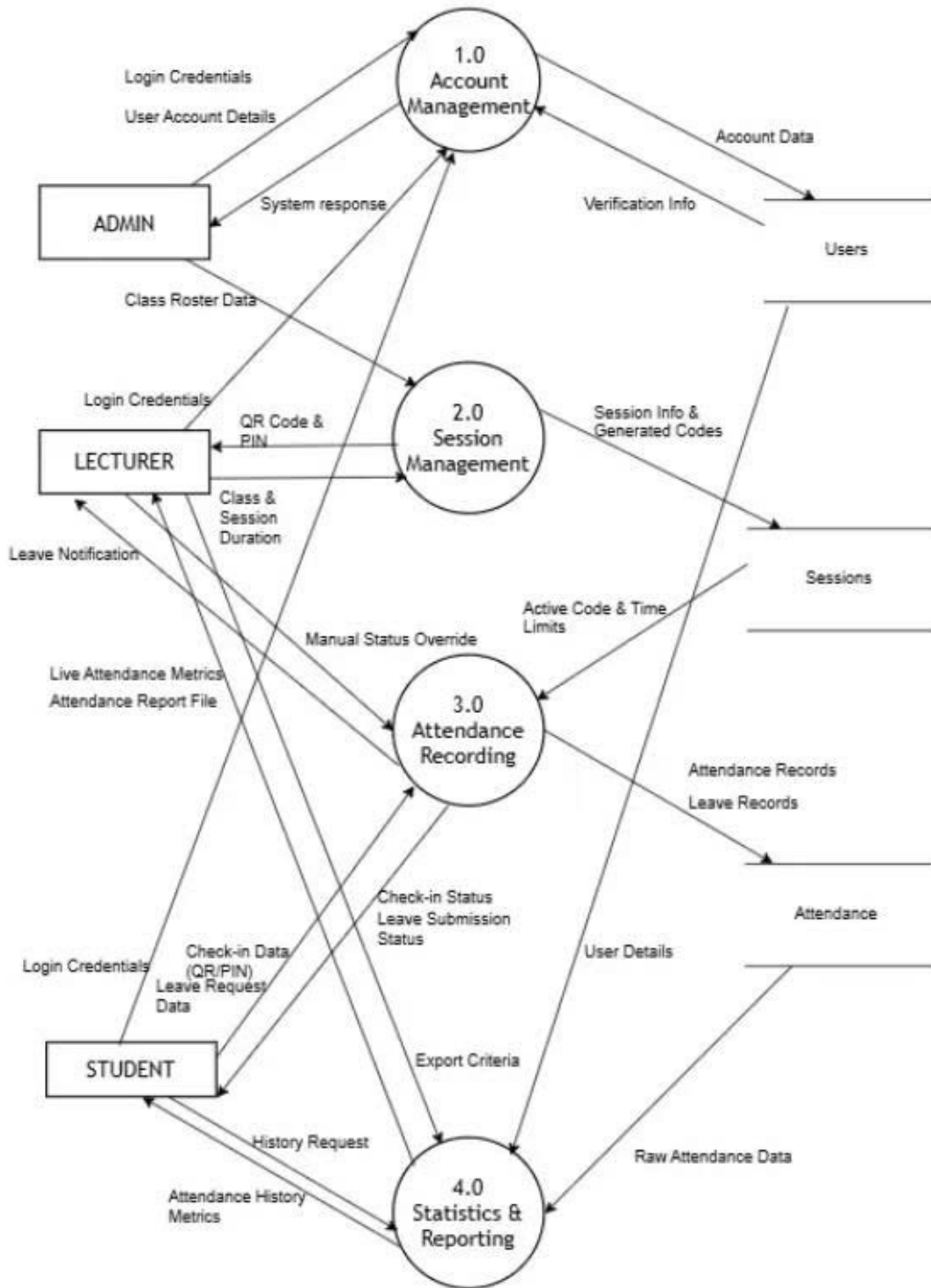


Figure 3.2: DFD Level 1 - Subsystem Functional Decomposition

Process 1.0 (Account Management): Handles credential authentication and profile lifecycle management against the Users data store.

Process 2.0 (Session Management): Manages class session creation, generates dynamic verification tokens, and stores active session metadata in the Sessions data store.

Process 3.0 (Attendance Recording): Receives check-in requests, validates tokens and timestamps against Sessions, updates the Attendance store, logs student leave submissions, and sends leave notifications to the Lecturer.

Process 4.0 (Statistics & Reporting): Queries the Attendance and Users stores based on lecturer/student requests to generate aggregated metrics and exportable spreadsheet reports.

4. Use Case Diagram & Specifications

4.1. Identification of Actors

Actor Name	Type	Description
Admin	Primary	System administrator managing user credentials, course configurations, and system access.
Lecturer	Primary	Academic instructor generating attendance sessions, monitoring live check-ins, and exporting summary reports.
Student	Primary	Enrolled learner checking in via QR/PIN, viewing attendance records, and submitting leave requests.
User	Abstract	Generalization base actor representing any authenticated identity in the system.

4.2. Identification of Use Cases

Use Case ID	Use Case Name	Primary Actor	Traceability (FR Mapping)
UC01	Login	User	FR7
UC02	Manage Accounts	Admin	FR1
UC03	Manage Classes	Admin	FR2
UC04	Create Attendance Session	Lecturer	FR3
UC05	View Real-time Attendance	Lecturer	FR4
UC06	Update Attendance Status	Lecturer	FR5
UC07	Export Attendance Report	Lecturer	FR6
UC08	Check-in Attendance	Student	FR8
UC09	View Attendance History	Student	FR9
UC10	Submit Leave Request	Student	FR10

4.3. Use Case Diagram

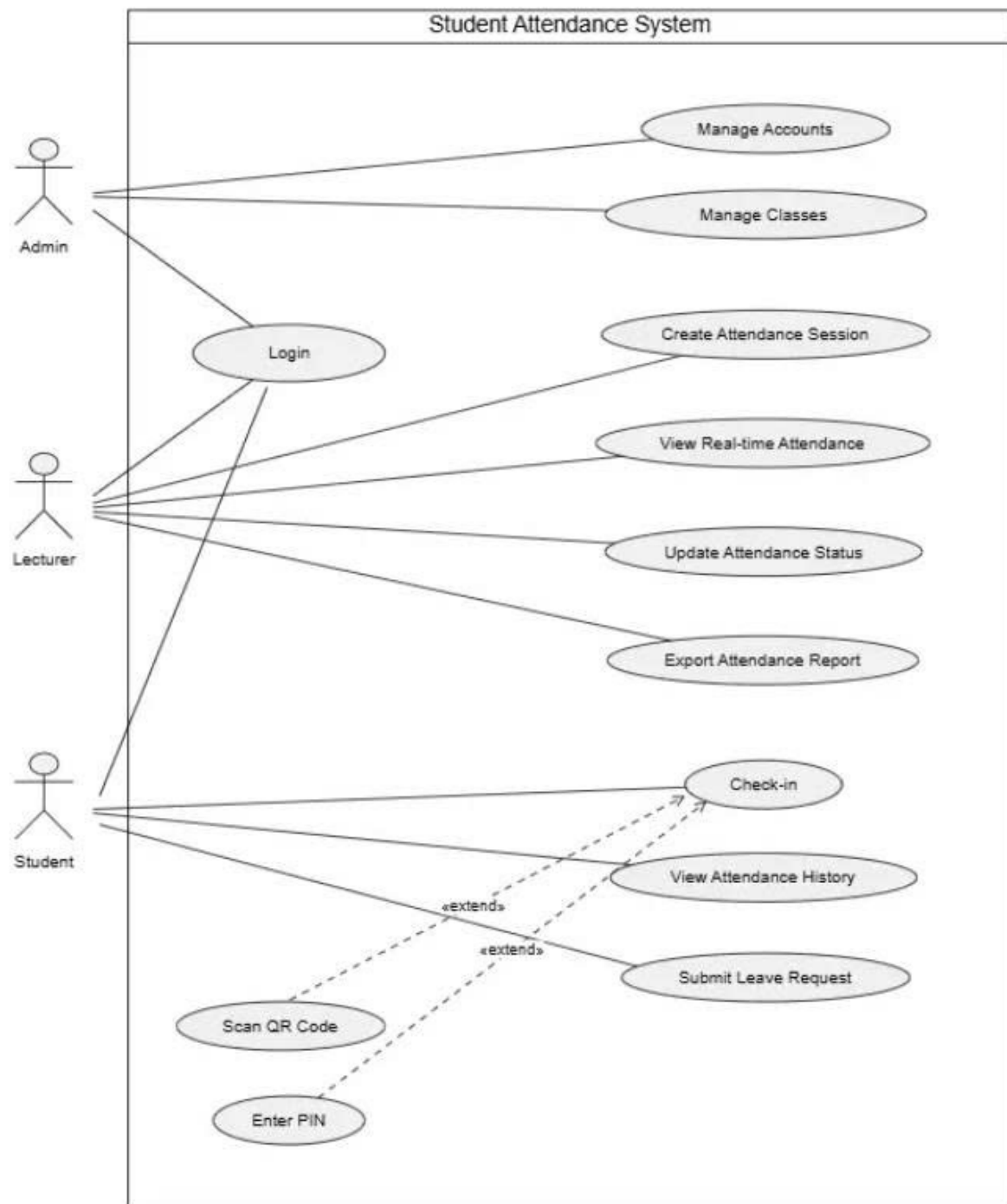


Figure 4.1: Use Case Diagram for Student Attendance System

4.4. Detailed Use Case Specifications

UC01: Login

Table 4.1a: UC01 - Specification Overview

Attribute	Details
Use Case ID & Name	UC01 - Login
Primary Actor	User (Admin, Lecturer, Student)
Summary Description	Allows registered users to authenticate their credentials and access their respective role-based dashboards.
Pre-conditions	The user account exists and is active in the database.
Post-conditions	The user is authenticated and redirected to their role-specific interface.

Table 4.1b: UC01 - Flow of Events

Flow Type	Step	Actor Action	System Response
Main Flow	1	Navigates to the login screen.	Renders login form with input fields.
	2	Enters credentials (Email/ID and Password) and clicks "LOG IN".	Validates credentials against hashed database records.
	3	Verification succeeds.	Generates session token and routes user to their dashboard.

Alternate Flow	2a	Enters invalid username or password.	Displays error prompt ("Incorrect Username or Password") and prompts retry.
Exception Flow	2b	Exceeds maximum failed login attempts (5 times).	Locks account temporarily for 15 minutes and logs security event.

UC04: Create Attendance Session

Table 4.2a: UC04 - Specification Overview

Attribute	Details
Use Case ID & Name	UC04 - Create Attendance Session
Primary Actor	Lecturer
Summary Description	Enables a lecturer to initialize an attendance tracking window, generating dynamic QR and 6-digit PIN codes.
Pre-conditions	Lecturer is authenticated; class section is assigned to the lecturer.
Post-conditions	Session status is set to "Active", dynamic tokens are generated, and the countdown timer begins.

Table 4.2b: UC04 - Flow of Events

Flow Type	Step	Actor Action	System Response
-----------	------	--------------	-----------------

Main Flow	1	Selects assigned class from dropdown menu.	Displays session configuration panel.
	2	Enters active duration (e.g., 15 minutes) and clicks "CREATE ATTENDANCE SESSION".	Validates duration; generates encrypted dynamic QR code and 6-digit PIN.
	3	Projects active session on screen.	Displays QR code, PIN, and starts real-time countdown timer.
Alternate Flow	2a	Enters invalid duration (e.g., ≤ 0 minutes).	Prompts lecturer to input a valid positive timeframe before proceeding.
Exception Flow	3a	Countdown timer reaches 00:00.	Automatically transitions session status to "Closed" and rejects subsequent check-ins.

UC08: Check-in Attendance

Table 4.3a: UC08 - Specification Overview

Attribute	Details
Use Case ID & Name	UC08 - Check-in Attendance
Primary Actor	Student
Summary Description	Allows enrolled students to log their attendance for an active session using either QR code scanning or manual PIN entry.

Pre-conditions	Student is authenticated; an attendance session for the class is currently "Active".
Post-conditions	Attendance transaction is saved in the database with status "Present" and recorded timestamp.
Extension Points	EP_InputMethod: Selection point between QR Code Scanning and Manual PIN Entry.

Table 4.3b: UC08 - Flow of Events

Flow Type	Step	Actor Action	System Response
Main Flow	1	Selects "Check-in" for the active class.	Displays submission options ("SCAN QR CODE" / "ENTER PIN").
	2	[EP_InputMethod] Clicks "SCAN QR CODE" and scans projected QR code.	Decodes token, validates session active status and TTL validity.
	3	Verification succeeds.	Records status as "Present" and displays success modal.
Extended Flow (3b)	2a	[EP_InputMethod] Clicks "ENTER PIN", types 6-digit PIN, and clicks "Check-in".	Validates PIN against active session; advances to Step 3.
Alternate Flow	2b	Submits expired or incorrect QR/PIN.	Displays error ("Code expired or invalid"); status remains "Absent".

Exception Flow	2c	Submits duplicate check-in for the same session.	Blocks transaction and alerts "Student already checked in".
-----------------------	----	--	---

UC07: Export Attendance Report

Table 4.4a: UC07 - Specification Overview

Attribute	Details
Use Case ID & Name	UC07 - Export Attendance Report
Primary Actor	Lecturer
Summary Description	Allows the lecturer to compile and download class attendance history into a spreadsheet file.
Pre-conditions	Lecturer is authenticated; target class contains historical attendance records.
Post-conditions	A formatted .xlsx report file is compiled and downloaded to the client device.

Table 4.4b: UC07 - Flow of Events

Flow Type	Step	Actor Action	System Response
Main Flow	1	Navigates to Attendance Report screen.	Displays course details, date filter, and summary table.

	2	Applies search/filter criteria and clicks "Export Excel".	Queries attendance records and compiles aggregate metrics.
	3	Selects download location.	Generates .xlsx file and initiates browser/desktop download.
Alternate Flow	2a	Filter query yields zero attendance logs.	Displays "No records found" warning and cancels file generation.

Specifications for Remaining Use Cases

Table 4.5: Specifications for UC02, UC03, UC05, UC06, UC09, UC10

Use Case ID & Name	Actor	Summary Description	Basic Flow Summary
UC02: Manage Accounts	Admin	Handles creation, updating, and deactivation of user accounts.	Admin navigates to User Management → selects action → inputs profile data → system validates and updates Users table.
UC03: Manage Classes	Admin	Creates course sections and assigns lecturers and students.	Admin opens Class Management → creates class → maps student/lecturer IDs → system saves mappings.
UC05: View Real-time Attendance	Lecturer	Monitors live check-in logs during an ongoing session.	Lecturer opens active session → system displays live student roster → auto-updates "Present" counter as check-ins arrive.

UC06: Update Attendance Status	Lecturer	Manually overrides student attendance status.	Lecturer searches student → changes status (e.g., Absent to Present) → saves → system updates AttendanceRecord.
UC09: View Attendance History	Student	Displays personal attendance metrics across enrolled classes.	Student opens "My Attendance" → selects course → system displays presence/absence totals and percentage.
UC10: Submit Leave Request	Student	Submits absence leave form with reason and proof.	Student opens form → inputs date and reason → attaches file → submits → system logs request as "Pending".

5. Class Diagram (OOP Design)

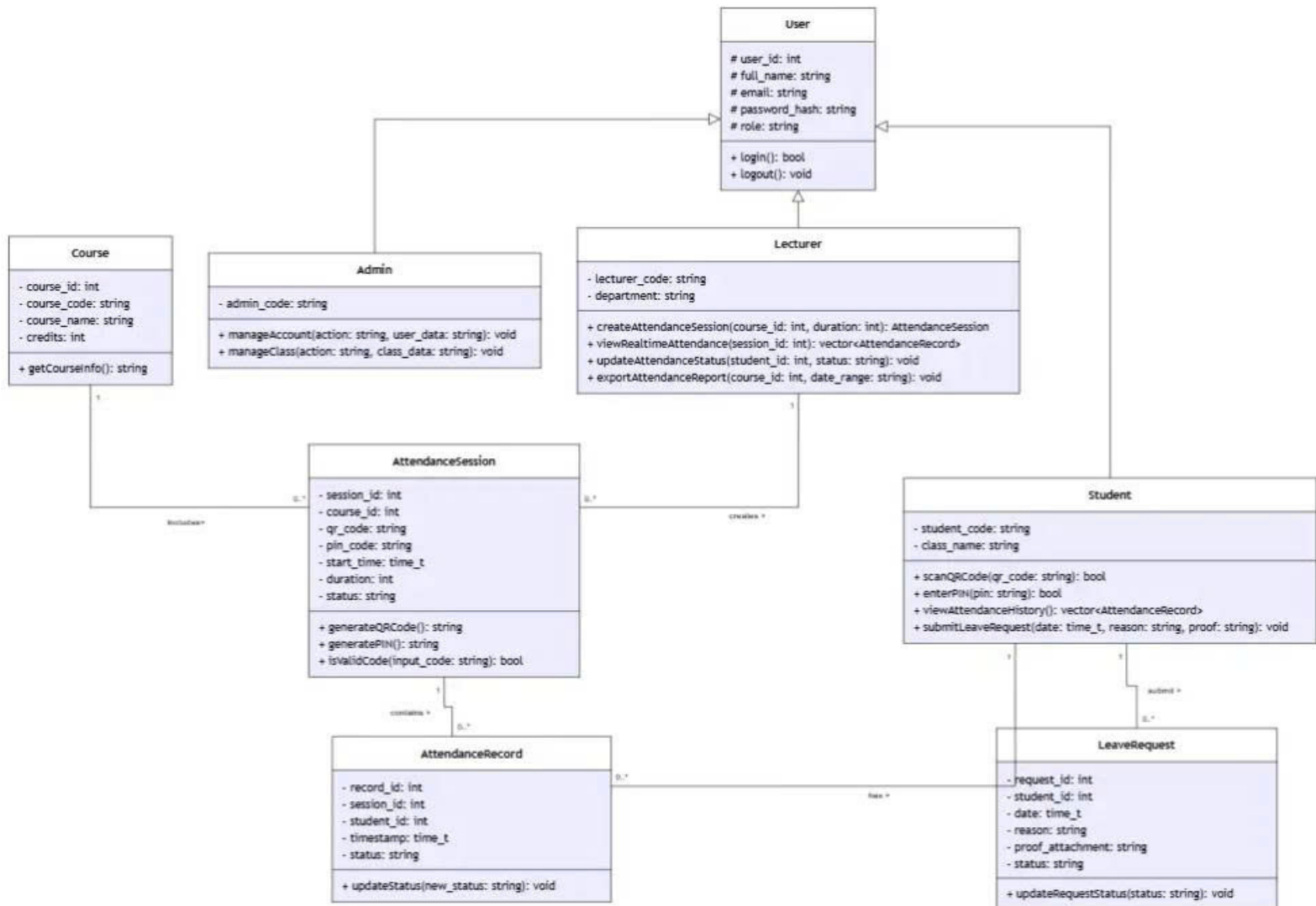


Figure 5.1: Object-Oriented Class Diagram of Student Attendance System

5.1. Class Descriptions and Responsibilities

User (Base Abstract Class): Encapsulates core authentication attributes and base operations (`login()`, `logout()`) shared by all system actors.

Admin (Derived Class): Inherits from User; provides administrative operations for managing user profiles (`manageAccount()`) and class rosters (`manageClass()`).

Lecturer (Derived Class): Inherits from User; manages session lifecycles (createAttendanceSession()), monitors active sessions (viewRealtimeAttendance()), overrides attendance states (updateAttendanceStatus()), and compiles reports (exportAttendanceReport()).

Student (Derived Class): Inherits from User; handles identity verification via dynamic tokens (scanQRCode(), enterPIN()), checks personal history (viewAttendanceHistory()), and submits absence requests (submitLeaveRequest()).

Course (Domain Class): Represents academic modules, storing course codes, names, credit units, and metadata retrieval methods (getCourseInfo()).

AttendanceSession (Domain Class): Manages dynamic check-in sessions, storing active tokens (qr_code, pin_code), timestamp boundaries, and validation algorithms (generateQRCode(), generatePIN(), isValidCode()).

AttendanceRecord (Association Class): Captures individual check-in transactions between a student and a session, recording exact verification timestamps and attendance statuses.

LeaveRequest (Domain Class): Encapsulates student absence applications, containing target dates, attached proof documents, and status evaluation operations (updateRequestStatus()).

5.2. Class Relationship & Multiplicity Specifications

Source Class	Target Class	Relationship Type	Multiplicity	Description
Admin	User	Generalization / Inheritance	—	Admin inherits common user attributes and authentication

				methods.
Lecturer	User	Generalization / Inheritance	—	Lecturer inherits base user attributes and extends role- specific logic.
Student	User	Generalization / Inheritance	—	Student inherits base user attributes and extends check- in operations.
Course	AttendanceSession	Association (includes >)	1 to 0.. *	One course curriculum includes zero to many attendance sessions.
Lecturer	AttendanceSession	Association (creates >)	1 to 0.. *	One lecturer can initialize zero to many attendance sessions.
AttendanceSession	AttendanceRecord	Association (contains >)	1 to 0.. *	One attendance session aggregates zero to many student attendance records.
Student	AttendanceRecord	Association (has >)	1 to 0.. *	One student possesses zero to many attendance

				history records.
Student	LeaveRequest	Association (submits >)	1 to 0.. *	One student can submit zero to many absence leave requests.

6. Data Model (Logical Database Schema)

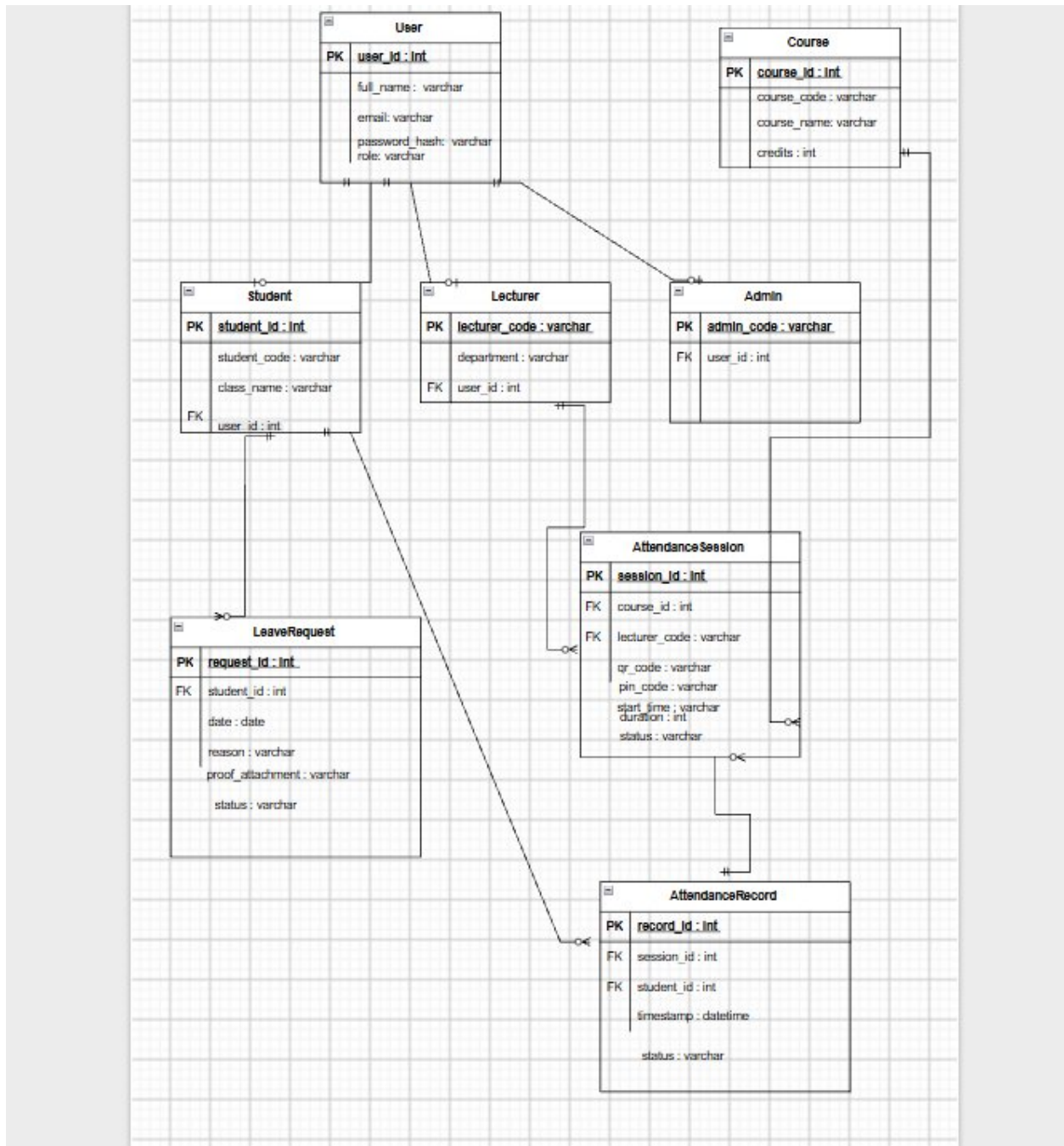


Figure 6.1: Logical Entity-Relationship Data Model

6.1. Entity Mapping Overview

Class Diagram Entity	Logical Relational Table
User	USER
Student	STUDENT
Lecturer	LECTURER
Admin	ADMIN
Course	COURSE
AttendanceSession	ATTENDANCE_SESSION
AttendanceRecord	ATTENDANCE_RECORD
LeaveRequest	LEAVE_REQUEST

6.2. Logical Table Schema Specifications

Table: USER

Column Name	Data Type	Key	Constraints
user_id	INT	PK	NOT NULL, AUTO_INCREMENT
full_name	VARCHAR(100)	—	NOT NULL
email	VARCHAR(100)	—	UNIQUE, NOT NULL

password_hash	VARCHAR(255)	—	NOT NULL
role	VARCHAR(20)	—	NOT NULL

Table: STUDENT

Column Name	Data Type	Key	Constraints
student_id	INT	PK	NOT NULL, AUTO_INCREMENT
student_code	VARCHAR(20)	—	UNIQUE, NOT NULL
class_name	VARCHAR(50)	—	NOT NULL
user_id	INT	FK	→ USER(user_id), NOT NULL

Table: LECTURER

Column Name	Data Type	Key	Constraints
lecturer_code	VARCHAR(20)	PK	NOT NULL
department	VARCHAR(100)	—	NOT NULL
user_id	INT	FK	→ USER(user_id), NOT NULL

Table: ADMIN

Column Name	Data Type	Key	Constraints
-------------	-----------	-----	-------------

admin_code	VARCHAR(20)	PK	NOT NULL
user_id	INT	FK	→ USER(user_id), NOT NULL

Table: COURSE

Column Name	Data Type	Key	Constraints
course_id	INT	PK	NOT NULL, AUTO_INCREMENT
course_code	VARCHAR(20)	—	UNIQUE, NOT NULL
course_name	VARCHAR(100)	—	NOT NULL
credits	INT	—	NOT NULL

Table: ATTENDANCE_SESSION

Column Name	Data Type	Key	Constraints
session_id	INT	PK	NOT NULL, AUTO_INCREMENT
course_id	INT	FK	→ COURSE(course_id), NOT NULL
lecturer_code	VARCHAR(20)	FK	→ LECTURER(lecturer_code), NOT NULL
qr_code	VARCHAR(255)	—	NULLABLE
pin_code	VARCHAR(6)	—	NULLABLE

start_time	DATETIME	—	NOT NULL
duration	INT	—	NOT NULL (in minutes)
status	VARCHAR(20)	—	NOT NULL

Table: ATTENDANCE_RECORD

Column Name	Data Type	Key	Constraints
record_id	INT	PK	NOT NULL, AUTO_INCREMENT
session_id	INT	FK	→ ATTENDANCE_SESSION(session_id), NOT NULL
student_id	INT	FK	→ STUDENT(student_id), NOT NULL
timestamp	DATETIME	—	NOT NULL
status	VARCHAR(20)	—	NOT NULL (Present/Late/Absent)

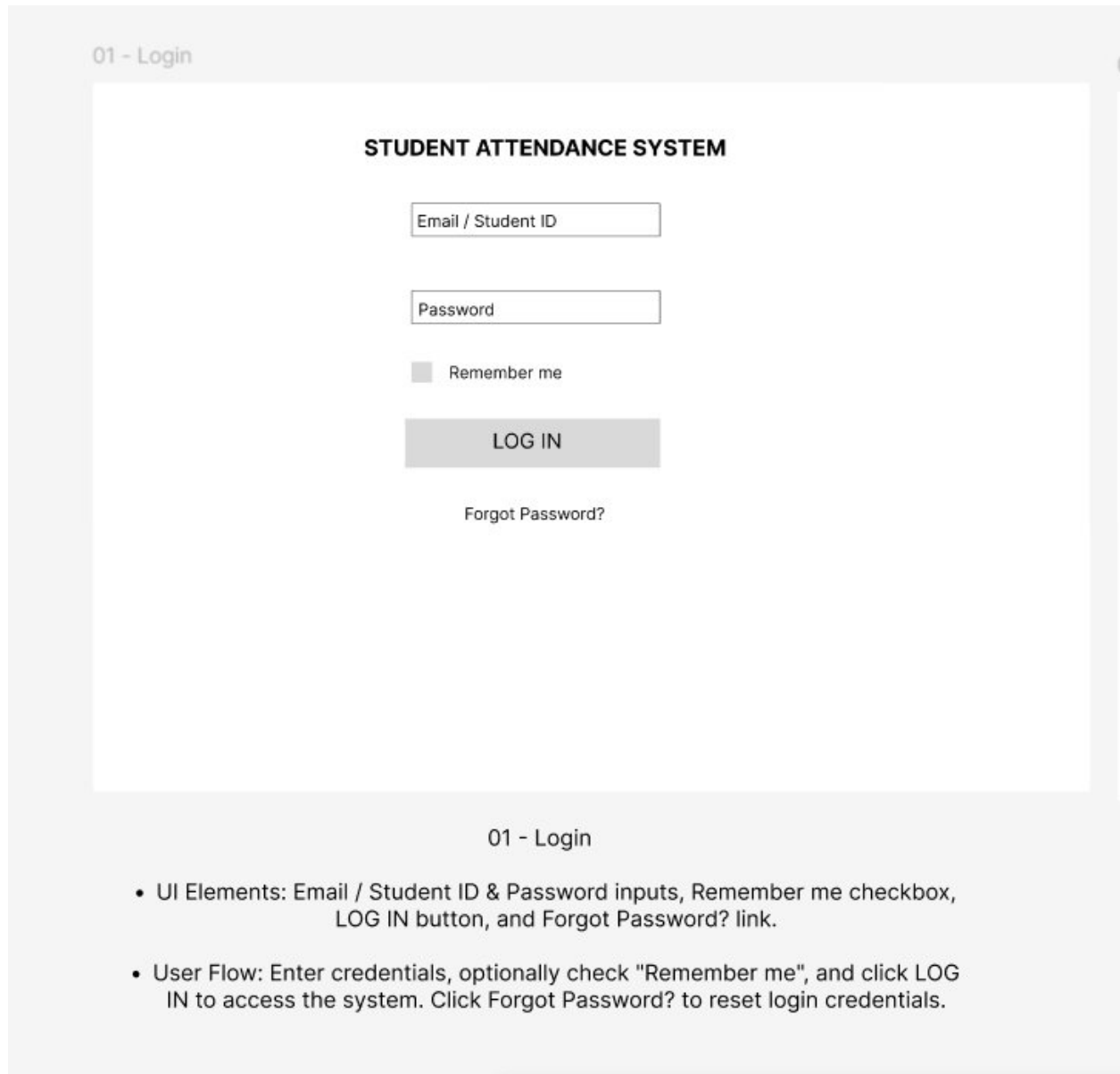
Table: LEAVE_REQUEST

Column Name	Data Type	Key	Constraints
request_id	INT	PK	NOT NULL, AUTO_INCREMENT
student_id	INT	FK	→ STUDENT(student_id), NOT NULL
date	DATE	—	NOT NULL

reason	VARCHAR(255)	—	NOT NULL
proof_attachment	VARCHAR(255)	—	NULLABLE
status	VARCHAR(20)	—	NOT NULL (Pending/Approved/Rejected)
course_id	INT	FK	→COURSE(course_id), NOT NULL

7. Interface Design Description

7.1. Screen 01 - Login



The wireframe shows a login interface for a 'STUDENT ATTENDANCE SYSTEM'. It includes a title bar '01 - Login', a central form area with input fields for 'Email / Student ID' and 'Password', a 'Remember me' checkbox, a 'LOG IN' button, and a 'Forgot Password?' link. Below the form, a legend explains the UI elements and user flow.

01 - Login

STUDENT ATTENDANCE SYSTEM

Email / Student ID

Password

☐ Remember me

LOG IN

[Forgot Password?](#)

01 - Login

- UI Elements: Email / Student ID & Password inputs, Remember me checkbox, LOG IN button, and Forgot Password? link.
- User Flow: Enter credentials, optionally check "Remember me", and click LOG IN to access the system. Click Forgot Password? to reset login credentials.

Figure 7.1: UI Wireframe - Screen 01: Login

UI Elements: Email / Student ID input field, Password input field, "Remember me" checkbox, "LOG IN" action button, "Forgot Password?" hyperlink.

User Flow: Users enter credentials into the respective input fields. Optionally, they can select "Remember me" to maintain session persistence. Clicking "LOG IN" verifies credentials against the database and opens the authorized dashboard. Clicking "Forgot Password?" opens the credential recovery interface.

7.2. Screen 02 - Lecturer Dashboard

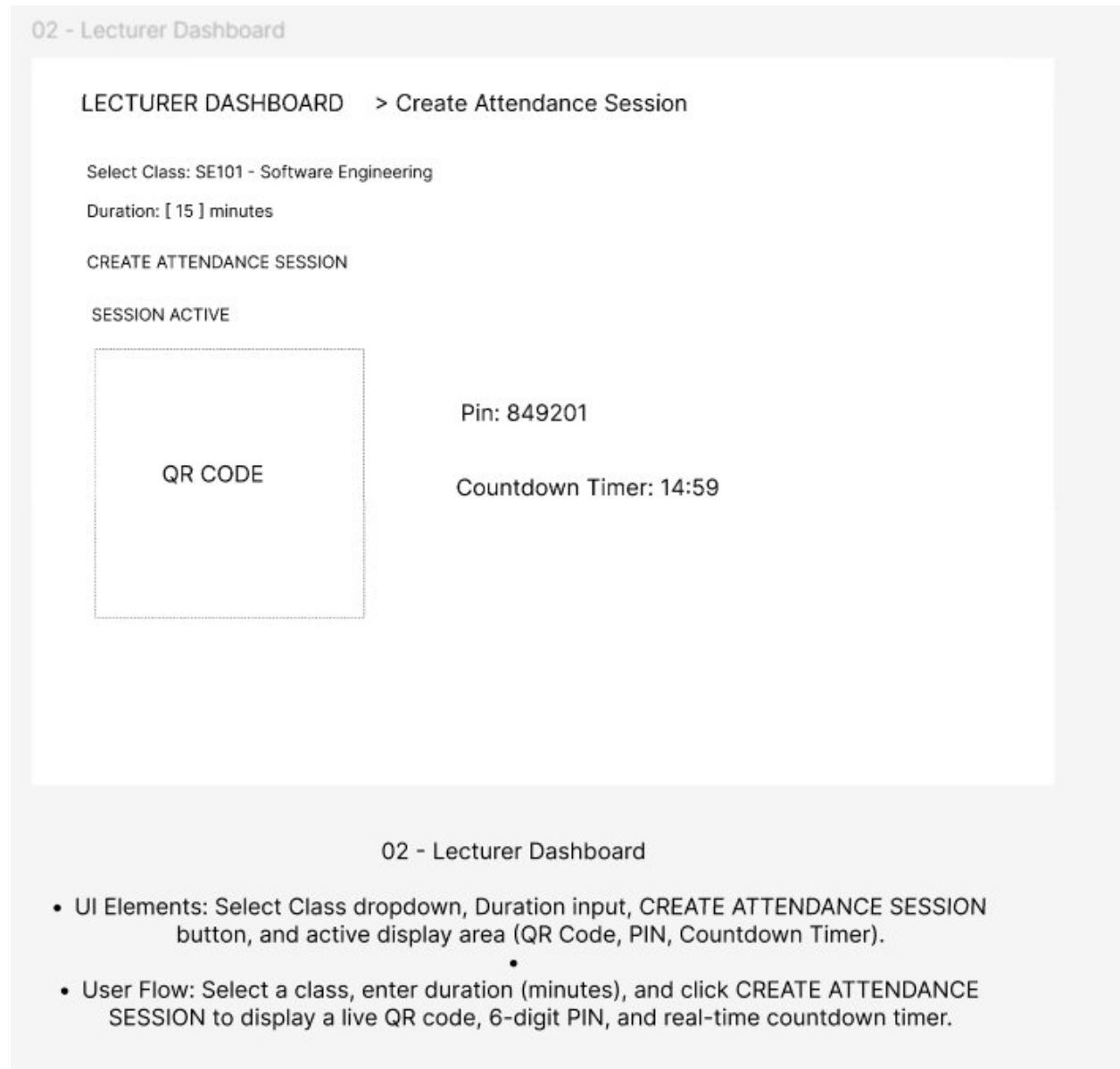


Figure 7.2: UI Wireframe - Screen 02: Lecturer Dashboard & Session Initialization

UI Elements: "Select Class" dropdown selector, "Duration (minutes)" input field, "CREATE ATTENDANCE SESSION" action button, active display viewport (Dynamic QR Code, 6-digit PIN, Countdown Timer).

User Flow: The lecturer selects an active class section (e.g., SE101 - Software Engineering) and enters the active timeframe in minutes. Clicking "CREATE ATTENDANCE SESSION" generates a dynamic QR code and 6-digit PIN on the display area while triggering the real-time countdown timer.

7.3. Screen 03 - Student Check-in

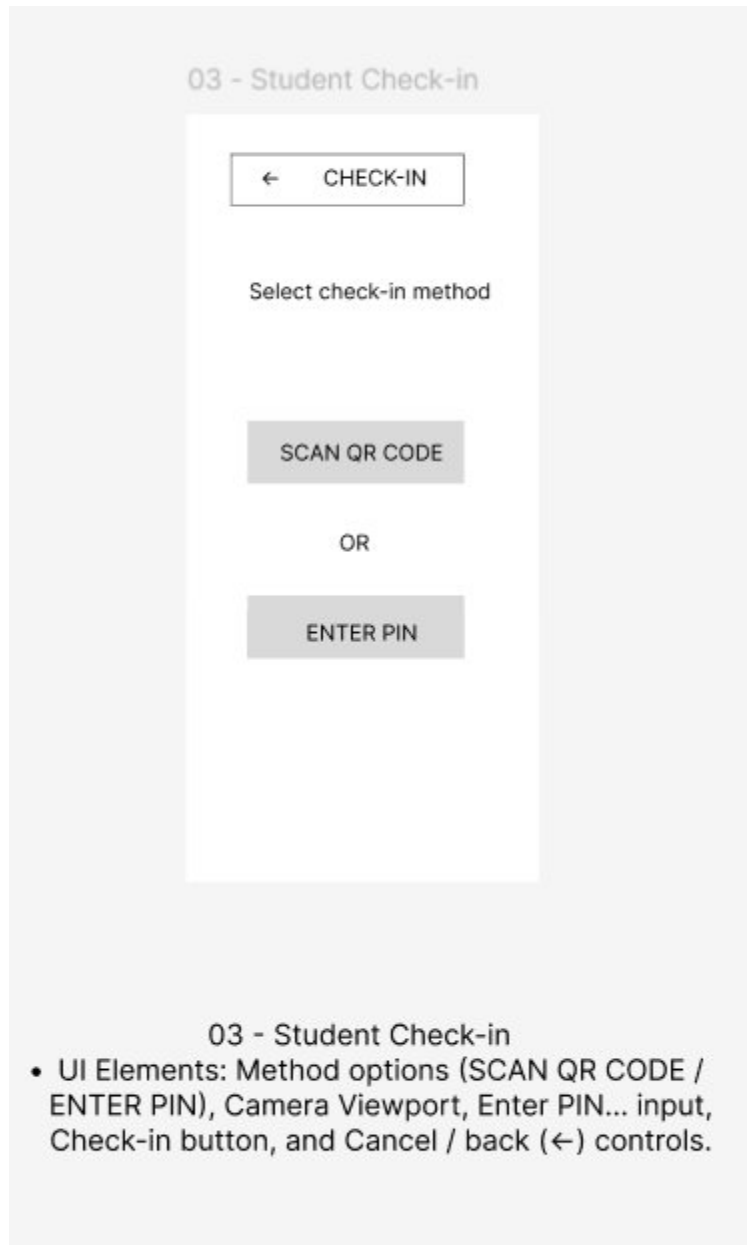


Figure 7.3: UI Wireframe - Screen 03: Student Check-in Options

UI Elements: Method options ("SCAN QR CODE" and "ENTER PIN" action buttons), camera viewport for scanning, PIN input field, "CHECK-IN" confirmation button, back navigation control (←).

User Flow: The student accesses the check-in screen on their mobile client and selects a submission mode. Choosing "SCAN QR CODE" activates the device camera to scan the projected QR code; choosing "ENTER PIN" allows manual 6-digit PIN input. Submitting valid input logs the attendance record.

7.4. Screen 04 - Attendance Report

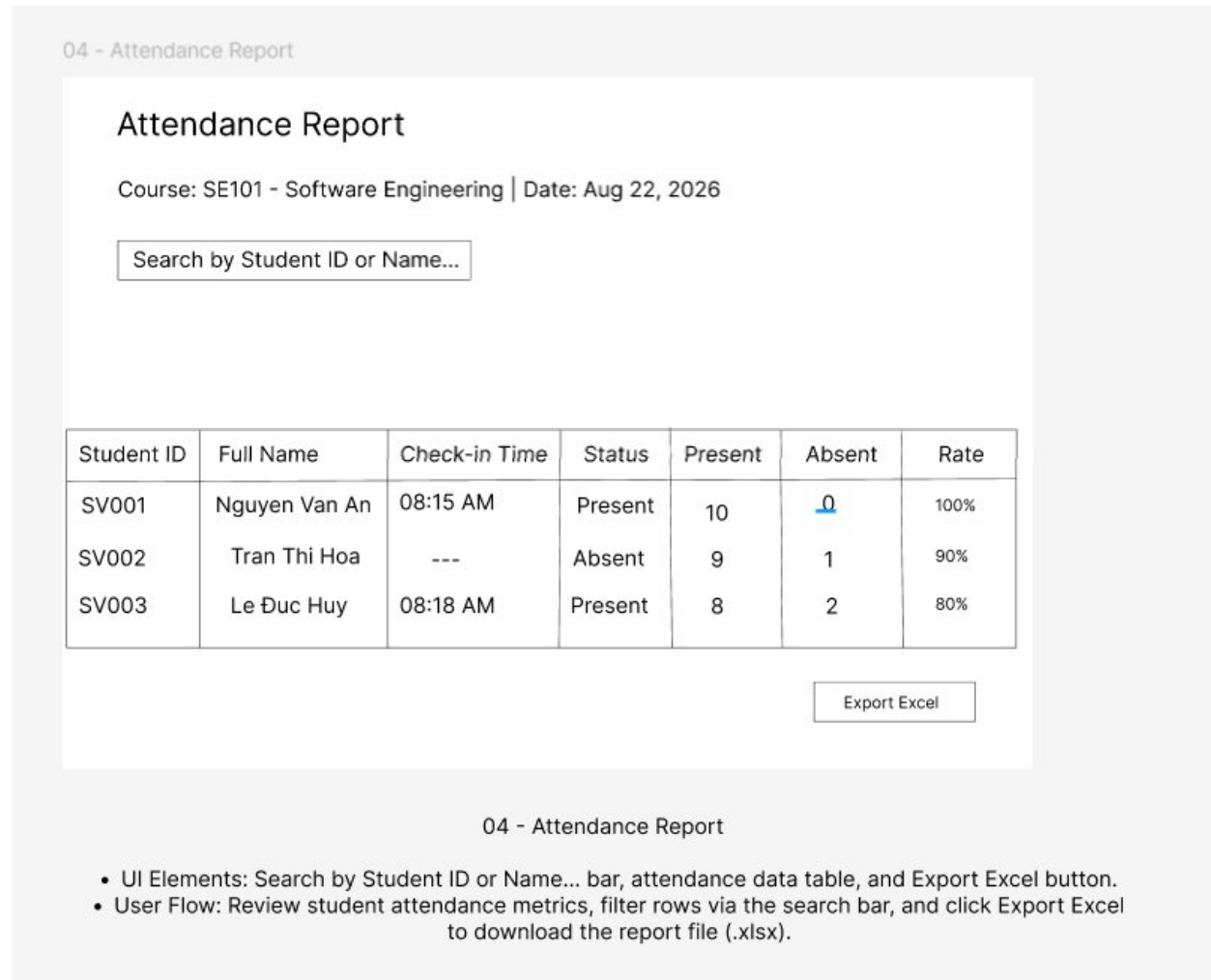


Figure 7.4: UI Wireframe - Screen 04: Class Attendance Summary Report

UI Elements: Course and Date header, "Search by Student ID or Name..." real-time filter bar, summary attendance data table (Columns: Student ID, Full Name, Check-in Time, Status, Present, Absent, Rate), "Export Excel" action button.

User Flow: The lecturer reviews aggregated class attendance metrics. Rows can be filtered in real time via the search bar. Interacting with the table allows manual status overrides, and clicking "Export Excel" downloads the structured spreadsheet report file (.xlsx).

GROUP 12 - TASK ASSIGNMENT & PROGRESS

No.	Full Name	Student ID	Progress
1	Phan Hữu Ý	056207011353	100%
2	Hồ Thiên Phú	052205010238	100%
3	Nguyễn Ngọc Như Quỳnh	082307003158	100%
4	Trịnh Nguyễn Kỳ Anh	089307004849	100%
5	Nguyễn Đặng Huyền Trâm	077307001119	100%